

02 — VALUE CATEGORIES, MOVE SEMANTICS & SPECIAL MEMBERS

This is the highest-yield modern-C++ interview area. Expect to reason about what moves, what copies, and why.

Take-home lesson. A "move" transfers *ownership of guts*, not bytes.

`std::move` moves nothing — it's a cast that grants *permission* to move.

The actual stealing happens in a move constructor/assignment you (or the library) wrote.

1. VALUE CATEGORIES (THE MENTAL MODEL)

Every expression has a type and a value category. Two properties: *has identity* (l) and *is movable* (m).

	has identity	no identity
movable	xvalue	----- prvalue
not-mov	lvalue	glvalue

- lvalue: has identity, can't be moved from implicitly. `x`, `*p`, `arr[i]`.
- prvalue: no identity, "pure value". `42`, `x+1`, `T()`, a non-reference return. In C++17 a prvalue *is* the initializer — it materializes into a temporary only when needed.
- xvalue: "expiring" — has identity but movable. `std::move(x)`, a function returning `T&&`, `arr[i]` where arr is `T&&`.
- glvalue = lvalue $\cup$ xvalue (has identity). rvalue = prvalue $\cup$ xvalue (movable).

Why it matters: overload resolution binds `T&&` to rvalues, `const T&` to anything. `std::move` is just a `static_cast<T&&>` — it *marks* an lvalue as movable, it does not move anything by itself.

Stop and think. Inside `void sink(std::string&& s)`, is `s` an lvalue or rvalue? What does `other = s;` do vs `other = std::move(s);`?

► answer

2. MOVE SEMANTICS

A move *steals* the guts of a source object (pointers/handles) and leaves it in a valid but unspecified state. Moves are an optimization, not a distinct operation — for trivial types a "move" is a copy.

```
class Buf {
    char* p_; size_t n_;
public:
    Buf(Buf&& o) noexcept : p_(o.p_), n_(o.n_) { o.p_=nullptr; o.n_=0; }
    // steal + null out
    Buf& operator=(Buf&& o) noexcept {
        if (this!=&o) { delete[] p_; p_=o.p_; n_=o.n_; o.p_=nullptr;
        o.n_=0; }
        return *this;
    }
};
```

Rules & gotchas:

- Mark move ops `noexcept`. `std::vector` reallocation uses moves *only if* the move ctor is `noexcept`; otherwise it copies (strong exception guarantee). This is a real, measurable perf cliff. `vector::push_back` growth is the canonical example.

```

struct Slow { std::string s; Slow(Slow&&) /* not noexcept */; };
struct Fast { std::string s; Fast(Fast&&) noexcept = default; };
// vector<Slow> growth COPIES every element on realloc; vector<Fast>
MOVES them.
// Verify with a static_assert so a future edit can't silently regress
you:
static_assert(std::is_nothrow_move_constructible_v<Fast>);

```

- Moved-from objects must be destructible and assignable — not necessarily usable otherwise. Don't assume it's empty; only assume it's valid.
 - A named rvalue reference is an lvalue. Inside `Buf(Buf&& o)`, `o` is an lvalue — you must `std::move(o.member)` to keep moving down the chain.
 - `const` kills moves: you can't move from a `const` object, so it copies. Returning `const T` by value is an anti-pattern (blocks move).
-

3. THE SPECIAL MEMBER FUNCTIONS & THE RULES

Six special members: default ctor, destructor, copy ctor, copy assign, move ctor, move assign.

Generation rules (memorize the interactions):

- Declaring any destructor, copy, or move op suppresses the *implicit* move operations.
- Declaring a copy op or destructor → move ops are not generated (you get copies as fallback). Deprecated-but-legal for copies.
- Declaring a move op → copy ops are deleted.
- `= default` to bring one back; `= delete` to forbid.

Rule of Three (pre-C++11): if you need a custom destructor, copy ctor, or copy assign, you almost certainly need all three (they manage a resource).

Rule of Five (C++11+): add move ctor + move assign.

Rule of Zero (preferred): design so you need *none* — hold resources in RAII members and let all six be compiler-generated correctly.

Copy-and-swap idiom — one function gives strong exception safety + handles self-assignment, at the cost of always constructing a copy:

```
T& operator=(T other) { swap(*this, other); return *this; } // by value = copy/move
```

4. COPY ELISION & RVO (C++17 MADE SOME MANDATORY)

- Mandatory copy elision (C++17): returning a prvalue, or initializing from a prvalue, elides the copy/move *guaranteed* — no copy/move ctor even needs to exist. `return T();` constructs directly in the caller's storage.
- NRVO (named RVO): returning a *named* local is *allowed* but not guaranteed to be elided. Still relies on a move/copy ctor existing.
- `return std::move(local)` is usually wrong — it *disables* NRVO by turning the return into an xvalue expression the compiler can't elide. Just `return local;`.

```
Widget make() { Widget w; ...; return w; } // NRVO candidate — do NOT std::move
Widget make() { return Widget{}; } // C++17: guaranteed elision
```

Take-home lesson. Trust the compiler to elide. Writing `std::move` on a return value is a pessimization that turns "zero copies" into "one move." The only time you `std::move` a return is when the local's type differs from the return type (e.g. returning a member, or a `unique_ptr` upcast).

5. PERFECT FORWARDING & FORWARDING REFERENCES

- `T&&` in a deduced context is a *forwarding (universal) reference*, not

an rvalue reference. `auto&&` too.

- Reference collapsing: `& & → &`, `& && → &`, `&& & → &`, `&& && → &&`.
- `std::forward<T>(x)` conditionally casts: preserves lvalue/rvalue-ness of the original argument. Use it exactly once per forwarded parameter.

```
template <class... Args>
auto make(Args&&... args) {                      // forwarding refs
    return std::make_unique<T>(std::forward<Args>(args)...);
}
```

Pitfalls: a forwarding-reference constructor can hijack overload resolution (greedily matches everything, beating copy ctor). Constrain it (`requires`, `enable_if`, or a `concepts` guard).

```
// C++20: stop the greedy forwarding ctor from eating copy/move
construction.
struct Person {
    std::string name;
    template <class T>
        requires (!std::same_as<std::remove_cvref_t<T>, Person>)    //
guard
        explicit Person(T&& n) : name(std::forward<T>(n)) {}
    Person(const Person&) = default;    // now actually reachable
    Person(Person&&) = default;
};
```

Stop and think. Without the `requires` guard, what does

`Person a{"x"}; Person b{a};` do?

► answer

DRILL PROMPTS

1. Given `void f(T&&)` and `void f(const T&)`, which binds for `f(x)`, `f(std::move(x))`, `f(T{})`?
2. Why does an un-`noexcept` move ctor make `vector` slower? Demonstrate.
3. Write Rule-of-Five for a class holding `char*`; then rewrite as Rule of Zero.

4. Explain why `return std::move(local)` pessimizes.

5. Implement `std::move` and `std::forward` yourself (they're one cast each).